

ソフトウェア設計及び実験 前期・個人開発企画書 そ〜だっしゅ！

学生番号 6125010806 氏名 清水 統貴

2026 年 6 月 24 日提出

1 ゲーム内容

1.1 タイトル

そ〜だっしゅ！

1.2 ジャンル

横スクロールアクションゲーム

1.3 コンセプト・概要

本ゲームはオリジナルキャラクターを操作してゴールを目指す横スクロールアクションである。最大の特徴は JoyCon を振ることでもたまる、体力も兼任している「炭酸ゲージ」を使い、ブーストすることで敵を倒しながらフィールド上を縦横無尽に高速飛行できるシステムである。

1.4 世界観

かつてシュワシュワで平和だった「炭酸王国」が、気の抜けた「ドロドロ帝国」によって侵略されてしまった。プレイヤーは炭酸王国の勇者「ソーダマン」となり、自らの炭酸パワーを弾けさせ、帝国軍をシュワっと浄化しながら駆け抜ける。

1.5 ルール

キャラクターを操作して、5 分以内にステージ右側のゴール地点に到達すればクリアとなる。道中で敵やキャップを集めるとスコアが上がる。時間切れ、通常状態で敵に接触、ステージに配置された穴に落ちる、消費のしすぎ等で「炭酸ゲージ」が 0 になるとゲームオーバーとなる。

1.6 UI

1.6.1 画面構成

ウィンドウサイズは 640 × 480 とする。

- メイン画面：キャラクターと横スクロールの地形
- UI 表示：画面左上にペットボトル型の「炭酸ゲージ」、画面右上にスコアとタイマーを配置する

画面構成のイメージを図 1 に示す。

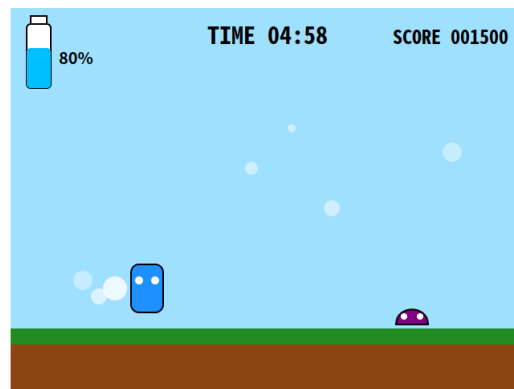


図 1 画面構成イメージ

1.6.2 操作方法

操作は JoyCon, PRO コントローラー, キーボードを使い, 下記に従う。

命令	内容
移動	アナログスティックまたは十字キーで左右移動. ゲージ使用時は全方向へ移動
ジャンプ	コントローラー使用時は右ボタン, キーボード使用時は space キー
ゲージの蓄積	Joy-Con, Pro コントローラーを上下に振る, キーボード使用時は W キーを連打
ゲージの使用, 解除	コントローラーの左ボタン, キーボード使用時は左シフトキー

2 実装方法

実装には SDL を用いる, SDL については, 授業資料を参考にする。

2.1 利用デバイス

Joy-Con, Pro コントローラー, キーボードを用いる. なお, Joy-Con または Pro コントローラー推奨である。

2.2 データ構造

現時点で必要だと考えられる変数, 構造体を示す (表 1) (表 2)

表 1 データ構造 (共通)

共通の構造		
データ型	変数名	
int	type	オブジェクトの型 (キャラ, アイテム, エフェクト等)
unsigned short	id	オブジェクトの識別番号
SDL_Point	pos	オブジェクトの座標 (x, y)
int	depth	描画の優先度 (深さ)

表 2 データ構造 (プレイヤー固有)

キャラ固有の構造		
データ型	変数名	
int	gauge	炭酸ゲージの蓄積量 (0~100)
int	state	現在の状態 (0: 通常時, 1: ブースト中)
unsigned int	score	獲得スコア
Uint32	start_time	ステージ開始時のシステム時刻 (SDL_GetTicks 等で取得)
int	time_left	残り時間 (秒). 0 になったら gauge を 0 にして終了

2.3 モジュール

現時点で必要だと考えられる関数をモジュールごとに示す (表 3).

- (1) ユーザインタフェース (gui_process.c)
- (2) 各モジュールの外部関数

3 ガントチャート

図 2 のようにガントチャートを作成した.

4 自己評価と理由

表 5 に示すように, すべて 2 点の合計 6 点で, 合格基準を満たしていると考え.

表3 ユーザインタフェース関係関数

void InitWindows(void)	
関数名	InitWindows()
機能	ウィンドウの初期化処理を行う関数
引数	なし
返り値	なし
Bool WindowEvent(void)	
関数名	WindowEvent()
機能	イベントを取得し、処理する関数
引数	なし
返り値	終了フラグ (True: 処理継続, False: 終了)

表4 外部関数

モジュールの外部関数			
関数名	機能	引数	返り値
InitSystem	ウィンドウと SDL, Joy-Con センサーの初期化	なし	成功/失敗フラグ
UpdateInput()	Joy-Con のセンサー値と ボタン入力の取得	なし	入力データ構造
UpdatePhysics()	重力処理, 地形・敵との当たり判定 (コリジョン) の計算	プレイヤー構造体へのポインタ	なし
DrawGraphics()	スプライト画像と UI の画面描画	描画対象のリスト	なし

タスク内容	第1週	第2週	第3週	第4週
企画・基本設計 (データ構造定義・素材準備)	■■■			
入力処理の実装 (Joy-Conセンサー・ボタン取得)	■■■			
物理演算・移動 (重力・バースト座標更新)	■■■	■■■		
当たり判定 (地形・敵キャラとのコリジョン処理)		■■■	■■■	
描画・UI実装 (キャラ・ゲージ・タイマー・背景)		■■■	■■■	
ゲーム進行管理 (タイトル→メイン→リザルト遷移)			■■■	
レベルデザイン (敵の配置・ステージ構成)			■■■	■■■
バランス調整・デバッグ (ゲージ消費量調整・バグ修正)				■■■

図2 ガントチャート

表5 自己評価と理由

項目	評価値	理由
ハードウェア構成	2	Joy-Con の加速度センサーを利用し、「振る」動作をゲーム進行に不可欠なゲージ蓄積の入力として実装する
技術面	2	センサー入力値のリアルタイムなスムージング処理と、横スクロールアクションにおける重力・当たり判定 (物理演算) を実装する
独創性	2	独自にデザインしたキャラクターのグラフィックとタイトルロゴを使用する。「振って力を溜め、一気に解放する」という独自のアクション性を提供する

5 参考にしたゲーム

スーパーマリオブラザーズを参考にした横スクロールアクションゲームを土台に、固有のシステムを追加した。

6 ゲームのアピールポイント

本ゲームのアピールポイントは以下の通りである。

- フィジカルと連携した爽快感：コントローラーを振って力をため、一気に開放する直感的なアクション
- 戦略的なリソース管理：炭酸ゲージ＝体力＝必殺技のエネルギー」という仕様により、バーストの使いすぎが自爆を招くハイリスク・ハイリターンの駆け引き
- 高いリプレイ性：5分という短い制限時間の中でクリアを目指す、テンポの良いゲームサイクル

参考文献

- [1] Google : Gemini <https://gemini.google.com/?hl=ja>